

ARCHITECTURE SPECIFICATION

Epistemic Runtime

A comprehensive specification of the Epistemic Runtime kernel: its four orthogonal primitives (Fact, Proof, Policy, Projection), the deterministic acceptance pipeline, production architecture, and the path from conceptual model to production-grade evidence kernel.

Version 0.8 · January 2026

Invariant-Enforced DAG · CRDT · ZK-Merge

Z.AI ARCHITECTURE SERIES

Table of Contents

Chapter 1: Introduction	3
Chapter 2: The Four Primitives	3
2.1 Fact: What Happened	4
2.2 Proof: Why We Believe It	4
2.3 Policy: Whether It Should Be Accepted	4
2.4 Projection: How Humans Consume It	5
Chapter 3: Core Architectural Insights	6
3.1 Primitives Are Orthogonal	6
3.2 State Is Derived, Not Accumulated	6
3.3 Policies Emit Facts	6
3.4 Identity Is Derived	7
Chapter 4: The Acceptance Engine	7
4.1 Canonicalize	7
4.2 Hash	7
4.3 Verify Schema	8
4.4 Verify Signatures	8
4.5 Evaluate Policies	8
4.6 Assign Logical Sequence	8
4.7 Persist	9
4.8 Emit Acceptance Fact	9
Chapter 5: Fact Lifecycle and Status	9
Chapter 6: Production Architecture	10
6.1 Applications Layer	11
6.2 Projection API	11
6.3 Projection Engine	11
6.4 Fact Log	12
6.5 Acceptance Engine	12

6.6 Adapters	12
Chapter 7: Architectural Gap Analysis	13
Gap 1: Acceptance Pipeline	13
Gap 2: Fact Status/Lifecycle	13
Gap 3: Deterministic Ordering	13
Gap 4: Canonical Serialization	14
Gap 5: Projection Versioning	14
Gap 6: Policy Time Travel	14
Gap 7: Proof Aggregation	14
Gap 8: Snapshot Semantics	14
Gap 9: Distributed Consensus	14
Gap 10: Failure Facts	15
Chapter 8: Long-term Stability	15
8.1 Failures as Facts	15
Chapter 9: Assessment Scorecard	16
9.1 Excellent Areas	17
9.2 Good Areas	17
9.3 Incomplete Areas	17
9.4 Missing Areas	17
Chapter 10: Conclusion and Next Steps	18

Chapter 1: Introduction

The Epistemic Runtime is a novel architectural framework designed to serve as the foundation of an evidence kernel: a system where every state change is observable, replayable, and verifiable. Unlike traditional event-sourcing systems that treat events as mere data records, the Epistemic Runtime treats every piece of information as an evidentiary artifact with cryptographic proof, declarative acceptance criteria, and deterministic derivation semantics. The result is a system where the boundary between what happened, why it is believed, whether it should be accepted, and how it is consumed is not just conceptual but architecturally enforced.

The problem the Epistemic Runtime solves is both fundamental and pervasive. In most distributed systems, state is accumulated through a series of mutations. When two nodes disagree on state, there is no principled way to resolve the conflict because the evidence trail is incomplete. When a bug is discovered, there is no way to replay the exact sequence of decisions that led to it. When an auditor asks "why was this record accepted?", the answer requires spelunking through logs, configuration files, and code history. The Epistemic Runtime eliminates these problems by making every decision a first-class, recorded, verifiable fact.

The design has evolved through multiple iterations. The original formulation used four primitives: Event, Identity, Evidence, and Fold. While this model captured the essential concerns, it suffered from overlapping responsibilities. Identity was both a primitive and derivable from Evidence; the relationship between Event and Fold was ambiguous. Through careful refactoring, the model arrived at the current formulation: Fact, Proof, Policy, and Projection. Each primitive owns exactly one orthogonal responsibility, and no two primitives share any conceptual overlap. This orthogonality is the key to the system's simplicity, replayability, and long-term stability.

The current production readiness of the Epistemic Runtime is estimated at approximately 80-85%. The conceptual model is mature and well-defined, the separation of concerns is excellent, and the event-sourcing and replayability semantics are production-quality. However, several critical gaps remain: the acceptance pipeline is not yet fully specified, deterministic ordering across distributed nodes needs work, projection versioning is planned but not implemented, and concurrency semantics are entirely missing. The purpose of this specification document is to capture the complete architecture as it stands, identify the remaining gaps, and provide a roadmap for reaching full production readiness.

Key Insight: *The Epistemic Runtime is not merely an event-sourcing framework. It is an evidence kernel where every state change carries cryptographic proof, every acceptance decision is recorded as a fact, and every derived state can be trivially replayed from the immutable Fact Log.*

Chapter 2: The Four Primitives

The Epistemic Runtime is built on four orthogonal primitives. Each primitive owns exactly one responsibility, and no two primitives share any conceptual territory. This design was not arrived at by accident; it is the result of multiple iterations that progressively eliminated overlaps and ambiguities. The four primitives are Fact, Proof, Policy, and Projection. Together, they form a closed system: every input to the runtime is a Fact, every justification for that Fact is a Proof, every decision about whether to accept it is governed by a Policy, and every way to consume the accumulated knowledge is a Projection.

2.1 Fact: What Happened

A Fact is the fundamental atom of the runtime. It captures a single, indisputable event in the system. Once created, a Fact can never be modified. Its identity is its content address: the SHA-256 hash of its canonical serialization. This means that two independent implementations producing the same logical Fact will always compute the same hash, enabling bit-exact verification across distributed nodes.

The canonical serialization follows RFC 8785 (JSON Canonicalization Scheme), which ensures deterministic byte representation regardless of key ordering, whitespace, or encoding differences. Every Fact is also schema-validated: its structure must conform to a declared schema before it enters the acceptance pipeline. This eliminates an entire class of malformed-input bugs that plague systems where validation is optional or inconsistent. The three defining properties of a Fact are: **Immutable** (once created, never modified), **Content-addressed** (SHA-256 hash of canonical serialization serves as identity), and **Schema-validated** (structure is verified before acceptance).

2.2 Proof: Why We Believe It

Proofs provide the evidentiary basis for Facts. A single Fact may be supported by multiple proofs: digital signatures (Ed25519), Merkle Mountain Range (MMR) membership proofs, zero-knowledge proofs of ancestry, or Trusted Execution Environment (TEE) attestations. The key insight is that proofs are composable: a proof may reference other proofs, creating a verification graph rather than a flat list. This composability is essential for building complex trust chains. For example, a Fact might be signed by key A, witnessed by an MMR root B, and verified by a TEE C. The combined proof provides a rich, multi-layered evidentiary basis that no single proof type could offer.

The three defining properties of a Proof are: **Cryptographic** (all proofs are mathematically verifiable, not based on trust or authority), **Composable** (proofs may reference other proofs, forming a directed acyclic verification graph), and **Verifiable** (any party with the appropriate public information can independently verify a proof without contacting the original author). This last property is critical for auditability: an auditor does not need to trust the system operator; they can verify the evidence themselves.

2.3 Policy: Whether It Should Be Accepted

Policies define the acceptance criteria for incoming Facts. They are authored in the .epd (Epistemic Policy Definition) DSL, versioned independently from the runtime, and evaluated at the time the Fact was accepted, never at the current policy version. This "policy time travel" is essential for replay: when re-evaluating historical facts, the system must apply the policies that were in effect at the time of original acceptance, not the policies that are current now. Without this, historical replay would produce different results depending on when it is run, which would violate the fundamental invariant of deterministic replay.

Critically, when a Policy rejects or corrects a Fact, it does not mutate state. Instead, it emits a correction Fact that is itself recorded in the Fact Log. This means Policy decisions are themselves observable, auditable, and replayable. Nothing disappears. The three defining properties of a Policy are: **Declarative** (policies are rules, not imperative code), **Time-bounded** (policies have effective dates and are always evaluated at Fact.acceptedAt), and **Emits correction facts** (rejection and correction produce new Facts, not exceptions or mutations).

2.4 Projection: How Humans Consume It

Projections transform the raw Fact Log into human-consumable state. The fundamental equation is: $\text{State}(t) = \text{Projection}(\text{Filter}(\text{FactLog}, \text{Policy}), t)$. Because state is always derived, never accumulated, the system supports trivial replay (re-run the projection), debugging (inspect any historical state), rollback (project at an earlier point), audit (show the full derivation chain), and simulation (project with hypothetical policies). Each of these capabilities falls out naturally from the derivation semantics without requiring any special-purpose machinery.

Projections are versioned with explicit schemas, ensuring that replaying historical facts with the projection logic that was active at the time produces identical results. The three defining properties of a Projection are: **Derived** (state is always computed from the Fact Log, never stored directly), **Versioned** (projection logic has explicit versions with schemas), and **Replayable** (re-running a projection with the same inputs always produces the same state). This versioning is critical for long-term correctness: without it, replaying 2026 facts with 2032 projection logic would silently change history.

Primitive	Responsibility	Key Property	Output
Fact	What happened	Immutable	Content hash
Proof	Why we believe it	Composable	Verification result
Policy	Whether to accept	Declarative	Acceptance / Correction Fact
Projection	How to consume	Derived	State(t)

Orthogonality Principle: *Nothing overlaps. Each primitive owns exactly one responsibility. Fact captures what happened, Proof justifies why we believe it, Policy decides whether to accept it, and Projection determines how to consume it. No two primitives share any conceptual territory.*

Chapter 3: Core Architectural Insights

Beyond the four primitives themselves, the Epistemic Runtime embodies several architectural insights that are not immediately obvious from the primitive definitions alone. These insights represent the distilled wisdom of the design iterations and are essential for understanding why the system works the way it does.

3.1 Primitives Are Orthogonal

Nothing overlaps. Each primitive owns exactly one responsibility. This is not accidental; it is the result of multiple design iterations. The original formulation used Event, Identity, Evidence, and Fold, which had overlapping concerns. Identity was both a primitive and derivable from Evidence. The relationship between Event and Fold was ambiguous. In the current model, all such overlaps have been eliminated. Fact owns "what happened," Proof owns "why we believe it," Policy owns "whether to accept it," and Projection owns "how to consume it." There is no shared territory, no ambiguity, and no need to decide which primitive handles a given concern. This orthogonality simplifies implementation, testing, and long-term evolution of the system.

3.2 State Is Derived, Not Accumulated

The equation $\text{State}(t) = \text{Projection}(\text{Filter}(\text{FactLog}, \text{Policy}), t)$ replaces the traditional $\text{State} += \text{mutation pattern}$. This single decision dramatically simplifies replay (re-run the projection), debugging (inspect any historical state), rollback (project at an earlier point), audit (show the full derivation chain), and simulation (project with hypothetical policies). In a traditional system, each of these capabilities requires special-purpose machinery: replay requires event logs and snapshots, debugging requires tracing infrastructure, rollback requires undo logs, audit requires separate audit trails, and simulation requires a sandbox environment. In the Epistemic Runtime, all five capabilities fall out naturally from the derivation semantics. The cost is computational: projections must be fast enough to recompute state from the Fact Log. But this cost is amortized through snapshotting and incremental projection, and the benefit in simplicity and correctness is enormous.

3.3 Policies Emit Facts

When a Policy rejects or corrects a Fact, it does not throw an exception or mutate state. Instead, it emits a correction Fact that is recorded in the Fact Log. This means Policy decisions are themselves observable, auditable, and replayable. Nothing disappears. The chain becomes: Policy evaluates the incoming Fact, produces a correction Fact if needed, the Projection incorporates the correction, and $\text{State}(t)$ reflects the correction. This design eliminates the "silent failure" problem that plagues most systems: when a record is rejected, there is no evidence trail, no way to understand why it was rejected, and no way to reconstruct the decision later. In the Epistemic Runtime, every policy decision is a Fact, and every Fact is permanent.

***The Policy-Emit Pattern:** Policy evaluates incoming Fact $\rightarrow$ Correction Fact is emitted $\rightarrow$ Projection incorporates correction $\rightarrow$ State(t) reflects the correction. The rejection itself becomes evidence.*

3.4 Identity Is Derived

In earlier versions of the system, Identity was a primitive. In the current model, identity is simply derived from proofs: Proof leads to Verifier, Verifier leads to Public Key, and Public Key leads to Identity. This removes an unnecessary abstraction layer and makes identity naturally composable. A Fact signed by key A, witnessed by MMR root B, and verified by TEE C has a rich identity derived from the composition of its proofs. There is no separate "identity primitive" that needs to be maintained, synchronized, or debugged. Identity is a projection over the proof graph, which means it inherits all the properties of projections: it is derived, versioned, and replayable. This simplification also eliminates an entire class of identity-related bugs, such as identity-key mismatches and stale identity caches.

Chapter 4: The Acceptance Engine

The Acceptance Engine is the actual kernel of the runtime. Without a deterministic acceptance pipeline, two nodes may disagree on which facts have been accepted, producing divergent state. The acceptance pipeline is an eight-stage process that transforms a raw Fact submission into a persistently recorded, fully verified Fact. Each stage is designed to be deterministic: given the same input, any two implementations must produce the same output. This determinism is the foundation of the system's consistency guarantees.

4.1 Canonicalize

The first stage applies RFC 8785 JSON Canonicalization Scheme to ensure a deterministic byte representation of the Fact. Without canonicalization, two nodes might serialize the same logical Fact differently (different key ordering, different whitespace), leading to different hashes and therefore different identities for the same Fact. The Canonicalizer interface abstracts the serialization format, allowing future migration to CBOR or other binary formats without changing the rest of the pipeline. This is a critical design decision: by isolating serialization behind an interface, the system avoids the "format coupling" that prevents many systems from evolving their wire format.

4.2 Hash

The second stage computes the SHA-256 hash of the canonical bytes, producing the Fact's content address. This hash serves as the Fact's immutable identity. Any change to the Fact, no matter how small, produces a completely different hash. The hash function is pluggable through the Canonicalizer interface, allowing future

migration to BLAKE3 or post-quantum hash functions. The key invariant is that two Facts with the same content hash are bit-identical: they contain exactly the same information, serialized in exactly the same way.

4.3 Verify Schema

The third stage validates the Fact's structure against its declared schema. This happens before any cryptographic work, because there is no point verifying signatures on a malformed Fact. Schema validation catches structural errors early, preventing wasted CPU cycles on invalid inputs. The schema is referenced by the Fact itself, which means the Fact declares what it should look like, and the engine verifies that it matches. Schema evolution is handled through versioning: a new schema version is a new schema, and Facts using old schemas continue to be valid.

4.4 Verify Signatures

The fourth stage performs Ed25519 signature verification. For each declared signer, the engine checks that the signer has produced a valid signature over the canonical hash. Signatures are verified against the canonical hash, not the raw bytes, which ensures that verification is independent of the serialization format. If any signature fails, the Fact is rejected with a `ProofVerificationFailed` correction Fact. The signature algorithm is also pluggable, allowing future migration to post-quantum signature schemes without changing the pipeline structure.

4.5 Evaluate Policies

The fifth stage runs the .epd policy engine against the Fact. The critical design decision here is that `Policy.effectiveAt` equals `Fact.acceptedAt`, never the current policy. This ensures that historical replay produces identical results regardless of when the replay is run. Policies may accept the Fact (it passes), reject the Fact (a rejection Fact is emitted), or emit correction Facts (the Fact is accepted with modifications recorded as separate Facts). The policy engine is declarative, not imperative: policies are rules, not code, which makes them auditable and predictable.

4.6 Assign Logical Sequence

The sixth stage assigns a deterministic ordering to the Fact. The ordering is: `LogicalSequence` first, then `Timestamp`, then `FactID`. This three-level comparison ensures total order across distributed nodes, even when Facts arrive out of order or with the same timestamp. The `LogicalSequence` is a monotonically increasing counter assigned by the consensus layer, the `Timestamp` provides a physical-time tiebreaker, and the `FactID` (which is the content hash) provides a final deterministic tiebreaker that is guaranteed to be unique.

4.7 Persist

The seventh stage writes the accepted Fact to the Fact Log. The Fact Log is append-only and content-addressed, which means that once a Fact is written, it can never be modified or deleted. The Fact Log is the source of truth for the entire system. Every Projection, every Policy decision, and every audit query ultimately derives from the Fact Log. The persistence layer is pluggable: implementations may use local disk, distributed storage, or content-addressed storage (such as IPFS) without changing the pipeline logic.

4.8 Emit Acceptance Fact

The eighth and final stage records the acceptance event itself as a Fact. This makes the acceptance pipeline observable: you can query the Fact Log to see not only what was accepted, but when it was accepted, what policies were applied, what proofs were verified, and what the outcome of each stage was. This observability is essential for debugging, auditing, and compliance. The Acceptance Fact includes the original Fact's hash, the policy version used, the verification results, and the assigned LogicalSequence.

The Canonicalizer interface that abstracts the serialization format:

```
interface Canonicalizer {  
    serialize(fact: Fact): bytes  
    deserialize(data: bytes): Fact  
    hash(canonical: bytes): Hash  
}
```

The deterministic ordering function that ensures total order:

```
Order(f1, f2) = compare(f1.logicalSequence, f2.logicalSequence)  
               || compare(f1.timestamp, f2.timestamp)  
               || compare(f1.factId, f2.factId)
```

Determinism Guarantee: Given the same input Fact, any two Acceptance Engine implementations must produce the same output (acceptance or rejection) and assign the same ordering. This is the foundation of cross-node consistency.

Chapter 5: Fact Lifecycle and Status

Facts are immutable, but their lifecycle status changes over time. These statuses are projections over metadata, not mutations to the Fact itself. This distinction is critical: the Fact Log contains only immutable Facts, and all lifecycle information is derived by projecting over the Fact Log. This means lifecycle history is fully auditable and replayable, just like everything else in the system.

Status	Description	Transition Trigger
Accepted	The Fact has passed through the Acceptance Engine and is persisted in the Fact Log.	Successful completion of all 8 stages
Rejected	The Fact failed policy evaluation or proof verification. A rejection Fact records the reason.	Policy rejection or proof verification failure
Superseded	A newer Fact replaces this Fact's authority. The supersession is recorded as a Fact.	A superseding Fact is accepted
Expired	The Fact's declared validity period has ended. Expiration is a projection over the Fact's timestamps.	System clock exceeds Fact.validUntil
Compensated	A compensating Fact has been recorded to undo the effect (similar to accounting reversal).	A compensating Fact is accepted

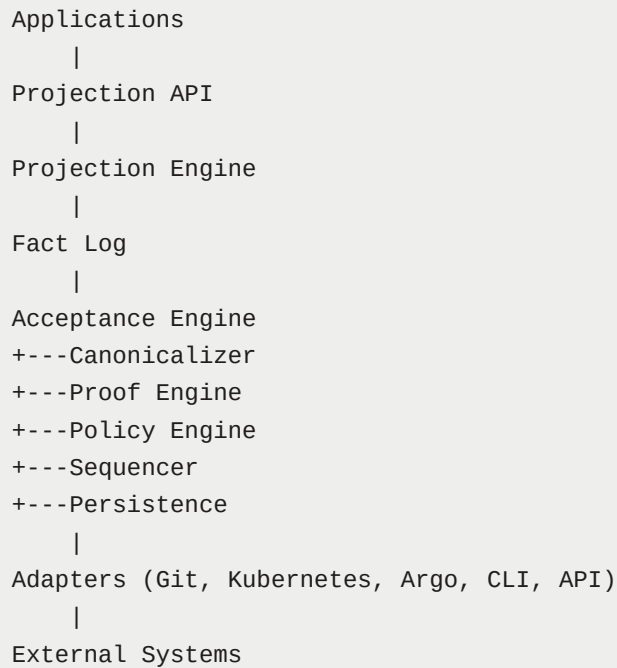
The valid lifecycle transitions are: Accepted may transition to Superseded (when a newer Fact replaces it) or Expired (when its validity period ends). Rejected may transition to Compensated (rare: when a rejection itself needs to be undone). Superseded may transition to Compensated (when the superseding action itself is reversed). Note that there is no transition from Rejected back to Accepted: once a Fact is rejected, it stays rejected, and the correct way to re-submit it is to create a new Fact with the corrected information.

The key principle is that the Fact itself never changes. Its lifecycle status is always a Projection over the Fact Log. This means that lifecycle history is fully auditable: you can see not only the current status of any Fact, but the complete chain of events that led to that status. You can also replay the lifecycle at any point in time, which is essential for debugging and compliance. The lifecycle status is computed by a special Projection that aggregates all Facts referencing a given Fact and derives the current status from the aggregation.

Immutability Invariant: *The Fact itself never changes. Lifecycle status is always a Projection over the Fact Log. This means lifecycle transitions are themselves Facts, and the complete lifecycle history is observable and replayable.*

Chapter 6: Production Architecture

The production architecture of the Epistemic Runtime follows a strict layered design. Each layer depends only on the layer below it, and no layer reaches across another. This layering ensures that changes to one layer (for example, swapping the persistence backend from local disk to distributed storage) do not affect any other layer. The architecture is designed for operational flexibility: the kernel is stable, while all infrastructure is replaceable.



6.1 Applications Layer

Applications are user-facing services that consume State(t) via the Projection API. They never interact with the Fact Log directly. This isolation means that application developers do not need to understand the internal architecture of the runtime; they only need to know how to query derived state. Applications may request state at a specific point in time (temporal queries) or with hypothetical policies (simulation queries), both of which are supported natively by the Projection API.

6.2 Projection API

The Projection API is a read-only interface that exposes derived state. It supports temporal queries (state at time T) and hypothetical queries (state with policy P). The API is versioned: each projection schema has an explicit version, and clients can request state in any supported version. This versioning ensures backward compatibility when projection schemas evolve. The Projection API is the only way applications interact with the runtime, which means all reads go through the same auditable pipeline.

6.3 Projection Engine

The Projection Engine evaluates projections over the Fact Log. It handles versioning: Projection v2.3 applied to 2026 facts must use v2.3 logic, not the current version. This is implemented by tagging each Fact with the projection version that was active at the time it was processed. The Projection Engine also supports incremental computation: when new Facts are appended to the Fact Log, only the affected portions of the state need to be recomputed. This makes projections efficient even for very long Fact Logs.

6.4 Fact Log

The Fact Log is the append-only, content-addressed log of all accepted Facts. It is the source of truth for the entire system. Every Projection, every Policy decision, and every audit query ultimately derives from the Fact Log. The Fact Log is implemented as an immutable, append-only data structure, which means it can be replicated across nodes without fear of divergence. Content addressing ensures that the same Fact always occupies the same position in the log, regardless of which node processed it.

6.5 Acceptance Engine

The Acceptance Engine orchestrates the eight-stage acceptance pipeline detailed in Chapter 4. It is not a fifth primitive; it is infrastructure. The key design principle is that the Acceptance Engine is deterministic: given the same input Fact and the same policy version, it always produces the same output. This determinism is what allows multiple nodes to process Facts independently and still agree on the resulting state. The Acceptance Engine delegates to the Canonicalizer, Proof Engine, Policy Engine, Sequencer, and Persistence components, each of which is a pluggable interface.

6.6 Adapters

Adapters are pluggable interfaces to external systems. The runtime provides adapters for Git (version control integration), Kubernetes (orchestration), Argo (continuous delivery), CLI (local development), and API (programmatic access). Adapters translate between the external system's protocol and the runtime's internal Fact/Proof/Policy model. The kernel never knows about external systems directly; all interaction goes through adapter interfaces. This isolation is essential for testability and portability.

The Consensus Adapter interface for distributed deployment:

```
interface ConsensusAdapter {  
  propose(fact: Fact): Promise<ConsensusResult>  
  getSequence(): LogicalSequence  
  subscribe(handler: FactHandler): Subscription  
}
```

Implementation	Consensus Model	Use Case
Raft	Strong consistency	Single-datacenter deployment with fault tolerance
BFT	Byzantine fault tolerance	Multi-party deployment where nodes may be adversarial
SingleNode	No consensus	Local development and testing

Chapter 7: Architectural Gap Analysis

Despite the conceptual maturity of the four-primitive model, several critical gaps remain before the Epistemic Runtime can be considered production-ready. This chapter catalogues ten identified gaps, each assessed for priority and impact. The gaps are ranked by a combination of criticality and the degree to which they block production deployment.

#	Gap	Status	Priority	Impact
1	Acceptance Pipeline	In Progress	Critical	Without it two nodes may disagree on accepted facts
2	Fact Status/Lifecycle	In Progress	High	Lifecycle changes without mutations
3	Deterministic Ordering	In Progress	High	Replay across nodes may diverge
4	Canonical Serialization	Planned	High	Serialization coupling prevents format migration
5	Projection Versioning	Planned	High	Replaying 2026 facts with 2032 logic changes history
6	Policy Time Travel	Planned	Critical	Historical replay becomes impossible
7	Proof Aggregation	Planned	Medium	Proof dependencies are lost
8	Snapshot Semantics	Planned	Medium	Snapshot lineage disappears
9	Distributed Consensus	Planned	Medium	Consensus coupling limits deployment
10	Failure Facts	Planned	High	Failures disappear without trace

Gap 1: Acceptance Pipeline

The acceptance pipeline is the most critical gap. Without a fully specified and implemented pipeline, two nodes may process the same Fact differently, leading to divergent state. The pipeline must be deterministic, and every stage must be specified with sufficient precision that independent implementations produce identical results.

Gap 2: Fact Status/Lifecycle

Facts have lifecycle statuses (Accepted, Rejected, Superseded, Expired, Compensated), but the mechanism for tracking these statuses without mutating the Facts themselves is not yet fully implemented. The lifecycle must be a Projection over the Fact Log, which requires a dedicated lifecycle projection engine.

Gap 3: Deterministic Ordering

The three-level ordering (LogicalSequence, Timestamp, FactID) is specified but not yet implemented. Without it, replay across nodes may diverge when Facts arrive in different orders. The ordering must be total and

deterministic, ensuring that any two nodes processing the same set of Facts produce the same ordered sequence.

Gap 4: Canonical Serialization

The current implementation couples serialization to the JSON format. The Canonicalizer interface is specified but not yet implemented, which means migrating to CBOR or other binary formats would require changes throughout the codebase. The interface must be implemented before any format migration can be considered.

Gap 5: Projection Versioning

Projection logic is currently versioned informally. Without explicit versioning and schema management, replaying 2026 facts with 2032 projection logic would silently change history. The system needs a projection registry that maps Fact timestamps to the projection version that was active at that time.

Gap 6: Policy Time Travel

Policies must be evaluated at `Fact.acceptedAt`, not at the current time. This is specified but not implemented. Without it, historical replay is impossible because the system would apply current policies to historical facts, producing different results than the original processing. This is a critical gap for audit and compliance.

Gap 7: Proof Aggregation

Proofs are composable, but the mechanism for tracking proof dependencies (a proof that references other proofs) is not yet specified. Without it, the verification graph is lost, and it becomes impossible to understand why a particular Fact is trusted. A proof graph data structure is needed to capture and query these dependencies.

Gap 8: Snapshot Semantics

Snapshots are needed for performance (avoiding full Fact Log replay on every projection), but the semantics of snapshots are not yet specified. In particular, snapshot lineage (which Facts were included in the snapshot, which projection version was used) must be recorded so that snapshots can be verified and invalidated when necessary.

Gap 9: Distributed Consensus

The ConsensusAdapter interface is specified, but the Raft and BFT implementations are not yet available. The current SingleNode implementation works for development but cannot be used in production. The consensus layer must also handle the interaction between LogicalSequence assignment and the consensus protocol.

Gap 10: Failure Facts

When the runtime encounters a failure (verification failure, policy violation, duplicate submission), the failure should be recorded as a Fact. Currently, failures are thrown as exceptions and disappear without trace. Failure Facts would make the system fully observable: every failure, its cause, and its context would be recorded in the Fact Log and queryable through Projections.

Chapter 8: Long-term Stability

The kernel can remain stable indefinitely if it consists only of the four primitives: Fact, Proof, Policy, and Projection. Everything else is replaceable infrastructure. This is not an aspiration; it is a design invariant. The four primitives define the ontology of the system, and the ontology is fixed. What can change are the implementations behind the interfaces: hash functions, signature algorithms, accumulators, orchestration platforms, version control systems, and continuous delivery tools.

Current	Future	Category
SHA-256	BLAKE3	Hash
Ed25519	Post-quantum signatures	Signatures
MMR	Verkle	Accumulator
Kubernetes	Nomad	Orchestration
Git	Perforce	VCS
Argo	Flux	CD

None of those changes alter the ontology. The four primitives remain constant. This is the key to long-term architectural stability: the kernel is minimal and orthogonal, while all implementation details are behind interfaces. When SHA-256 is eventually replaced by BLAKE3, the change is isolated to the Canonicalizer interface. When Ed25519 is replaced by a post-quantum signature scheme, the change is isolated to the Proof Engine. The Fact, Policy, and Projection primitives are unaffected.

8.1 Failures as Facts

Instead of throwing exceptions, the runtime produces failure facts. These failures become evidence. They are recorded in the Fact Log, observable in projections, and auditable like any other fact. This approach transforms failures from ephemeral events (visible only in logs that are periodically rotated and deleted) into permanent records that can be queried, analyzed, and correlated with other facts in the system.

Failure Type	Description	When It Occurs
FactRejected	The Fact was rejected by policy evaluation	Policy engine returns reject
ProofExpired	A proof has exceeded its validity period	Proof validUntil is in the past
PolicyViolation	The Fact violates one or more active policies	Policy evaluation finds violations
DuplicateFact	A Fact with the same content hash already exists	Content hash collision in Fact Log
ReplayConflict	Replay produces a different result than the original	State(t) divergence detected
ConsensusFailure	The consensus protocol failed to reach agreement	Consensus timeout or partition
ProjectionFailure	The projection engine failed to compute state	Projection schema mismatch or error

Stability Through Minimality: The four primitives are the fixed kernel. Everything else (hash functions, signature algorithms, consensus protocols, storage backends) is replaceable infrastructure behind well-defined interfaces. The system can evolve without breaking the ontology.

Chapter 9: Assessment Scorecard

The following scorecard summarizes the current production readiness assessment of the Epistemic Runtime. Each area is scored on a 0-100 scale and assigned a qualitative grade. The overall production readiness is estimated at approximately 80-85%, driven primarily by the strength of the conceptual model and the significant gaps in distributed systems, storage, and concurrency.

Area	Score	Grade
Conceptual model	95	Excellent
Separation of concerns	95	Excellent
Event sourcing	95	Excellent
Replayability	95	Excellent
Cryptographic model	80	Good
Distributed systems	40	Incomplete
Version evolution	35	Incomplete
Storage model	40	Incomplete

Area	Score	Grade
Concurrency	10	Missing
Production readiness	82	~80-85%

9.1 Excellent Areas

The conceptual model, separation of concerns, event sourcing, and replayability areas all score 95 (Excellent). These areas represent the core strength of the Epistemic Runtime. The four-primitive model is clean, orthogonal, and well-specified. The derivation semantics ($\text{State}(t) = \text{Projection}(\dots)$) are elegant and powerful. The policy-emit pattern ensures that all decisions are observable. These areas are already production-quality and require no fundamental changes, only continued attention to implementation quality and test coverage.

9.2 Good Areas

The cryptographic model scores 80 (Good). The model is sound: Ed25519 signatures, SHA-256 hashing, and MMR membership proofs are all well-understood and correctly applied. However, the Canonicalizer interface is not yet implemented, which means the system is coupled to JSON serialization. Algorithm agility (the ability to swap hash and signature algorithms without changing the pipeline) is specified but not implemented. Advancing this area to Excellent requires implementing the Canonicalizer interface and adding algorithm negotiation to the Proof Engine.

9.3 Incomplete Areas

Distributed systems (40), version evolution (35), and storage model (40) all score Incomplete. These areas need specification work before implementation can begin. Distributed systems needs the ConsensusAdapter implementations (Raft, BFT) and the interaction between LogicalSequence assignment and consensus. Version evolution needs the projection registry and the policy time-travel mechanism. Storage model needs specification of the persistence interface, the snapshot semantics, and the compaction protocol for long-running Fact Logs.

9.4 Missing Areas

Concurrency scores 10 (Missing). This is the most significant gap in the current specification. The system needs a complete concurrency model that specifies how multiple concurrent Fact submissions are handled, how optimistic and pessimistic concurrency control interact with the append-only Fact Log, and how concurrent projection updates are coordinated. This area needs a design from scratch, informed by the existing literature on concurrent event-sourcing systems and CRDT-based conflict resolution.

Chapter 10: Conclusion and Next Steps

The Epistemic Runtime represents a significant advancement in the design of evidence-based systems. The progression from the original Event/Identity/Evidence/Fold formulation to the current Fact/Proof/Policy/Projection model is a clear improvement: the ontology is simpler, responsibilities are cleaner, and the system is more replayable. The key architectural insights (orthogonal primitives, derived state, policies that emit facts, derived identity) are not merely theoretical; they have practical implications that simplify implementation, testing, and operation.

The remaining work is not about inventing new concepts but about specifying execution semantics. The four primitives are well-defined; what is missing is the machinery that makes them work in practice. The acceptance pipeline must be fully specified and deterministically implemented. Policy time travel must ensure that historical replay produces identical results. Projection versioning must prevent silent history changes. Canonical serialization must isolate the system from format coupling. Each of these gaps is well-understood and has a clear path to resolution.

With the identified additions, the architecture would be suitable as the foundation of a production-grade evidence kernel. The priority roadmap is as follows: the Acceptance Pipeline comes first, because without it, two nodes cannot agree on accepted facts. Policy Time Travel comes second, because without it, historical replay is impossible. Projection Versioning comes third, because without it, replaying old facts with new logic silently changes history. The remaining items follow in dependency order.

Priority	Item	Depends On
1	Acceptance Pipeline	Canonicalizer interface
2	Policy Time Travel	Policy versioning registry
3	Projection Versioning	Projection schema registry
4	Canonical Serialization	Canonicalizer interface
5	Fact Lifecycle	Lifecycle projection engine
6	Failure Facts	Acceptance Pipeline
7	Deterministic Ordering	ConsensusAdapter
8	Proof Graph	Proof aggregation engine
9	Snapshots	Snapshot lineage engine
10	Consensus	Raft/BFT implementation

The Path Forward: *The architecture is sound. The four primitives are stable. The remaining work is execution: specifying, implementing, and testing the machinery that makes the primitives work in a distributed, concurrent, production environment. Each gap has a clear resolution path, and the priority ordering ensures that the most critical capabilities are delivered first.*